Case 02 · Design document: Fire-Sale Cascade Simulator

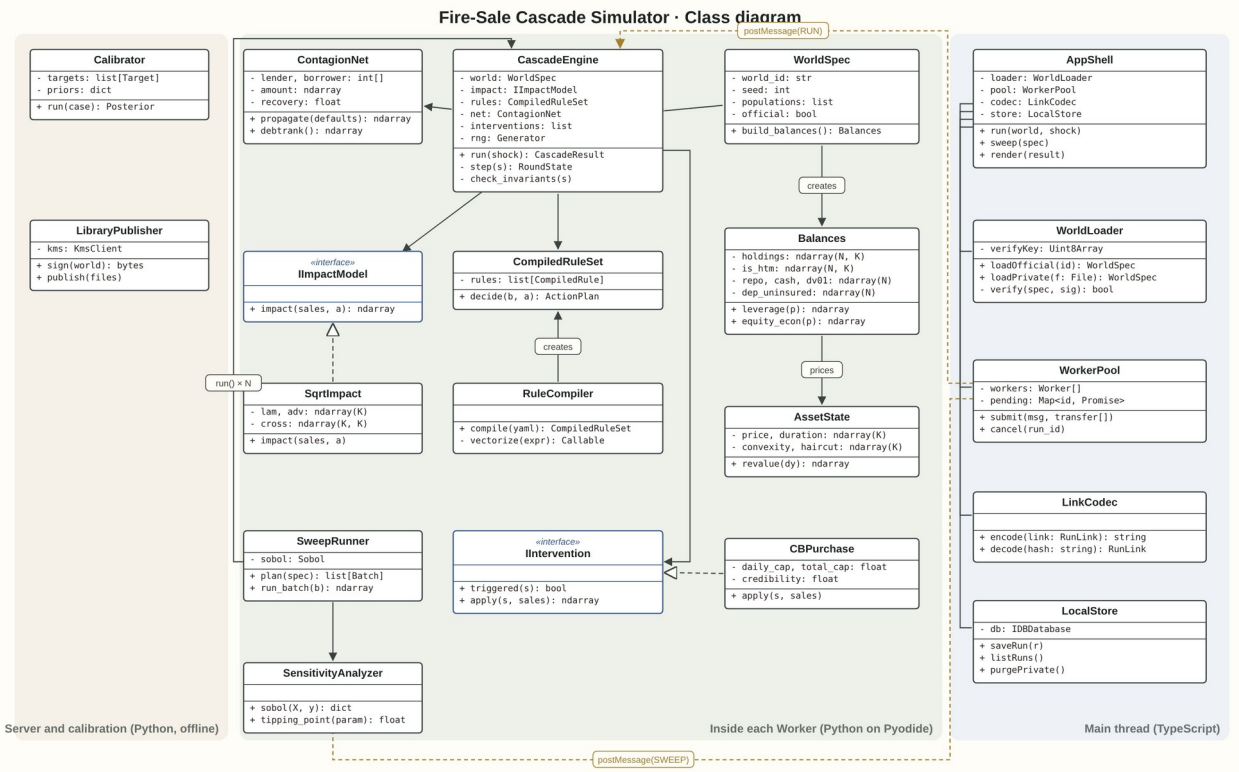
The stage after architecture. The architecture decided which components exist and what passes between them. Design goes one level down: which classes live inside each component, which states each object passes through, who talks to whom and in what order, which threads run in parallel, and what the tables look like. The structure matches the Case 01 design document and the Trivia server document.

Design principles

Four principles, taken directly from the patterns chosen in the architecture:

- **The engine is pure:** CascadeEngine.run() takes a world, a shock and a seed and returns a result. It makes no call to the network, the clock, IndexedDB or the DOM. Everything that touches the outside world lives in the main thread (Functional Core, Imperative Shell).
- **An agent is an index, not an object:** there is no Agent class. There is Balances, in which every attribute is an array over all agents, and every step of a round is a vector operation over the arrays (Data-Oriented Design).
- **An interface for everything that changes between studies:** the price-impact model and the intervention are interfaces (IImpactModel, IIntervention), because those are exactly what researchers will want to swap. Rules are data, not code (Interpreter).
- **No result without a range:** a single run is never shown alone. The main thread sends a default sweep after it, and only when that returns is the result shown with p10, p50 and p90.

Class diagram



Three areas, and three runtimes. On the right, the browser's main thread in TypeScript: it manages the interface, loading and saving, and computes nothing. In the middle, what runs inside each Worker: Python on Pyodide, with NumPy. On the left, what runs on the server and not at the user: calibration and signing.

The dashed lines are postMessage: there is no direct call between the main thread and the engine. This is also what makes it possible to run exactly the same engine in the calibration cluster, with no browser at all.

The role of each class

Class	Area	Role	What it does not do
AppShell	Main	Takes input, starts runs and sweeps, draws	Computes nothing in the model
WorldLoader	Main	Loads a world from the library or a private file, checks schema and Ed25519 signature	Never sends a private file anywhere
WorkerPool	Main	Holds the Workers, distributes work and merges results	Does not know what is inside a message
LinkCodec	Main	Encodes a run definition into the URL fragment and back	Encodes only the definition, never balance sheets
LocalStore	Main	Stores runs, sweeps and private worlds in IndexedDB	Does not sync to the server
CascadeEngine	Worker	The round loop: revalue, decide, settle, intervene, impact, contagion	Does not know what an "LDI fund" is
WorldSpec	Worker	The world's description: populations, parameters, seed, official or not	
Balances	Worker	All agents' balance sheets as arrays (N agents, K assets)	Has no decision logic
AssetState	Worker	Price, duration, convexity and haircut for each asset; revaluation by yield change	
IImpactModel	Worker	Interface: net sales in, price change out	
LinearImpact, SqrtImpact, DealerCapacity	Worker	The three implementations: linear Kyle, square root, and market makers' balance-sheet limit	
RuleCompiler	Worker	Translates YAML into vector functions, once per run	Never runs eval on input
CompiledRuleSet	Worker	Returns an ActionPlan: how much each agent sells, posts or withdraws	
IIntervention	Worker	Interface: triggered(state)	

Class	Area	Role	What it does not do
		and apply(state, sales)	
CBPurchase, CollateralRelief, DepositGuarantee, ParFacility	Worker	The four tools from the spec: central-bank purchases, collateral relief, deposit insurance, a par-value facility	
ContagionNet	Worker	A graph of direct debts; a default's loss passes to lenders; DebtRank up front	
SweepRunner	Worker	Runs a segment of the Sobol sequence and returns an array of outcomes	
SensitivityAnalyzer	Worker	Sobol indices (SALib) and a tipping point for each parameter	
Calibrator	Server	ABC-SMC (pyabc) against target data from a historical case	Does not run in the browser
LibraryPublisher	Server	Signs worlds and calibrations and publishes them as static files	The private key never leaves the KMS

Message protocol

As with the Trivia server, every message between the main thread and a Worker is defined in advance: a type field, a run identifier, and content. The difference: there is no network here, there is postMessage, and large arrays move as Transferables so they are not copied.

```
→ Worker   { type: "RUN",   run_id, engine_ver, spec: WorldSpec,
             buffers: { holdings, repo, cash, ... }  [transfer],
             shock, interventions[], seed }
→ Worker   { type: "SWEEP", run_id, batch_idx, base_spec, params: Float64Array
[transfer] }
→ Worker   { type: "CANCEL", run_id }
← Worker   { type: "PROGRESS", run_id, done, total }
← Worker   { type: "RESULT",   run_id, rounds: RoundState[] (typed arrays), converged:
bool, summary }
← Worker   { type: "SWEEP_RESULT", run_id, batch_idx, outcomes: Float64Array [transfer] }
← Worker   { type: "ERROR",    run_id, code, detail }
```

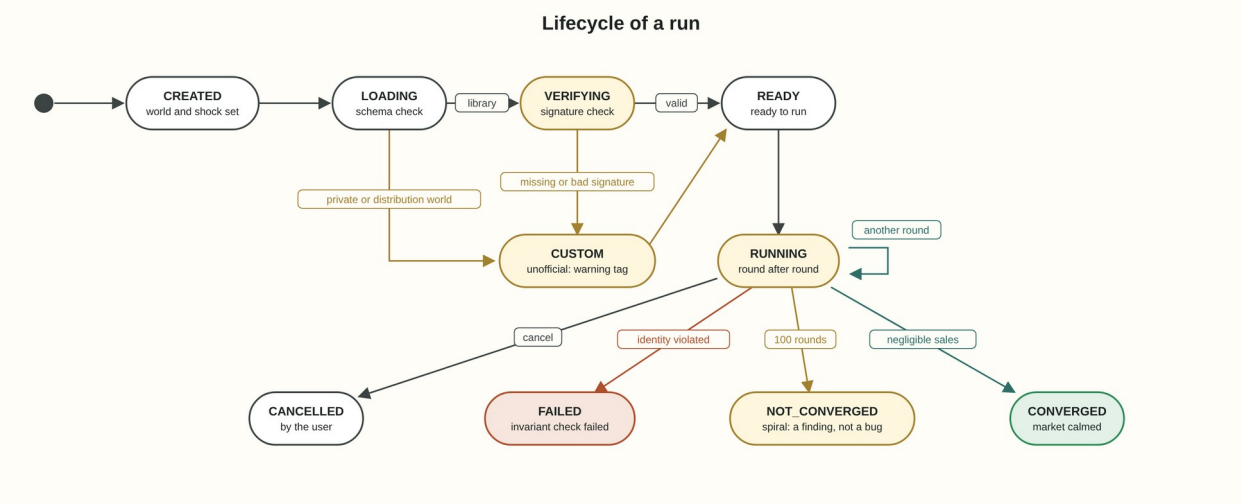
Error code	When	What is shown
E_SCHEMA	The world does not match the schema (a missing field, a negative number)	The offending field, and nothing runs
E_RULES	Invalid YAML, or a rule using an action that does not exist	The line in the file
E_INVARIANT	A conservation check failed: assets do not equal liabilities plus equity, or a sale exceeds the	The round, the agent and the gap. This is a bug, not a finding

Error code	When	What is shown
	holding	
E_RESOURCE	The Worker exceeded memory or time	A suggestion to shrink the world

Notice what is not an error: a cascade that has not converged after 100 rounds comes back as a RESULT with converged: false. A spiral that does not end is a finding, and the interface presents it as one.

State machines

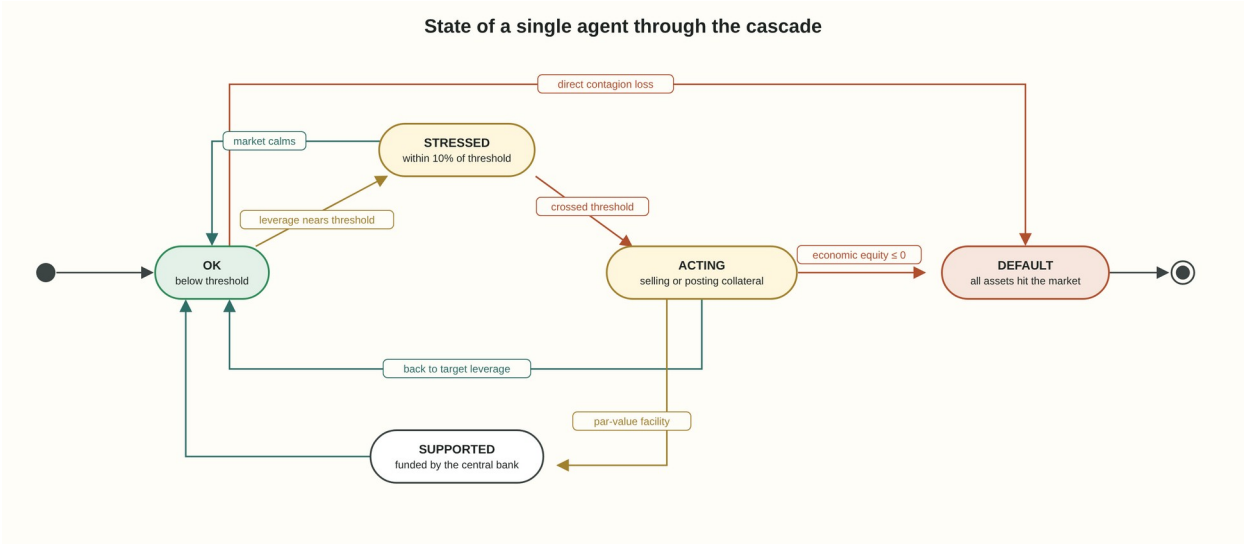
Lifecycle of a run



The run is the object the user sees, and every state it passes through is shown in the interface.

- VERIFYING happens only for a library world. A private world or one built from distributions goes straight to CUSTOM and receives an "unofficial" badge that cannot be removed.
- A bad signature does not stop the run. It demotes it to CUSTOM. This is deliberate: the user can still explore, but no screenshot will present the result as official.
- Four final states, and only one of them (FAILED) is a fault. NOT_CONVERGED is shown as a result in its own right, highlighted.

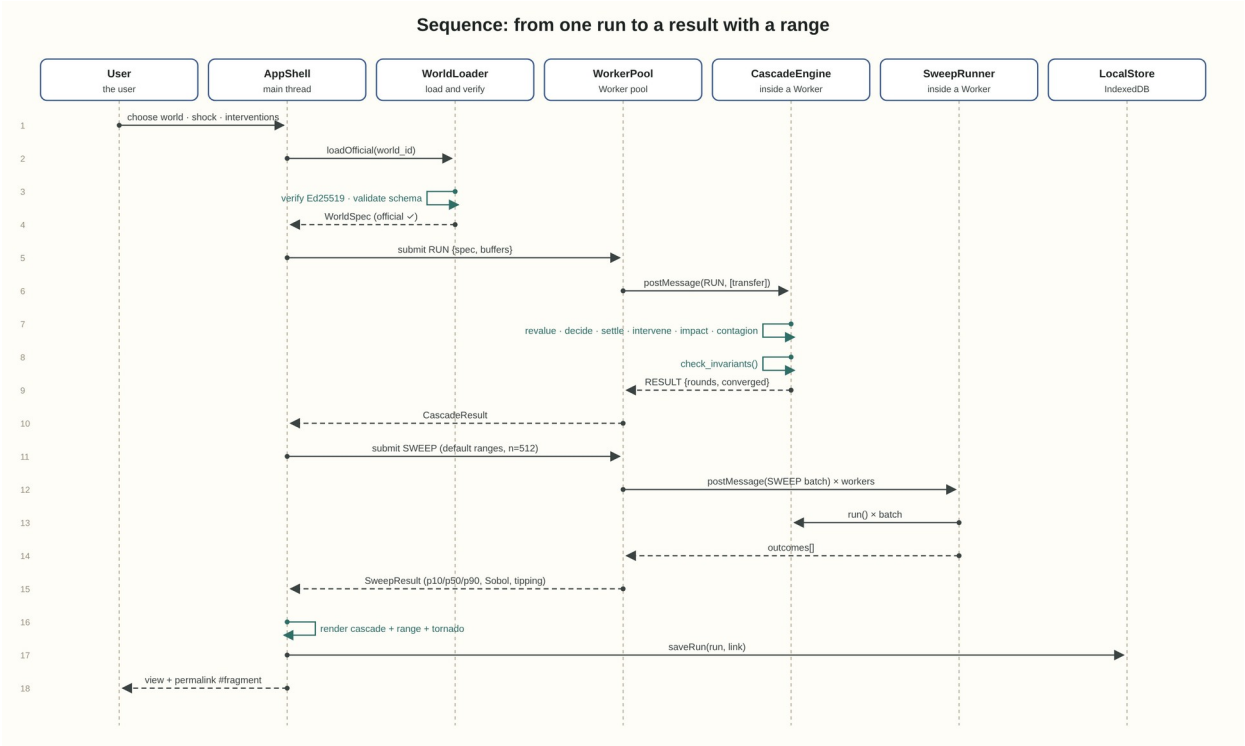
State of a single agent



This machine does not exist as an object: it is computed each round as an array of states, one value per agent. It exists for display and understanding: it explains to the user why an agent sold.

- STRESSED is a warning state only, within 10% of the threshold. It does not change behaviour, but it is what is marked in yellow in the view, and it shows how close the market is to the tipping point.
- **The path to DEFAULT can bypass everything else:** a direct contagion loss (another agent failed and did not repay a loan) can bring down an agent that was in a healthy state.
- SUPPORTED exists only when a par-value facility is configured. An agent in this state does not sell, so it does not move the price. That is exactly the mechanism by which an intervention breaks the loop.

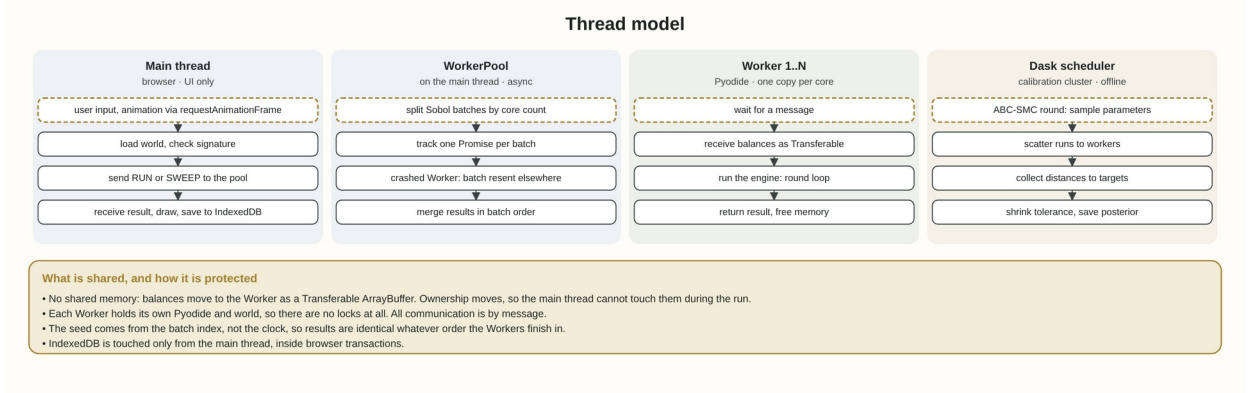
Sequence diagram



The sequence shows a single user action, end to end. Three points:

- **Steps 5 and 6:** the balance sheets move as Transferables. After sending, the main thread can no longer access them. That is not a bug; it is the guarantee that no two threads share the same memory.
- Step 11 happens automatically, without the user asking. It is the principle "no result without a range" translated into code.
- **Step 17:** the run is saved locally, and the link is a fragment (after #). Browsers do not send the fragment to the server, so even the run definition reaches nowhere the user has not sent it themselves.

Thread model

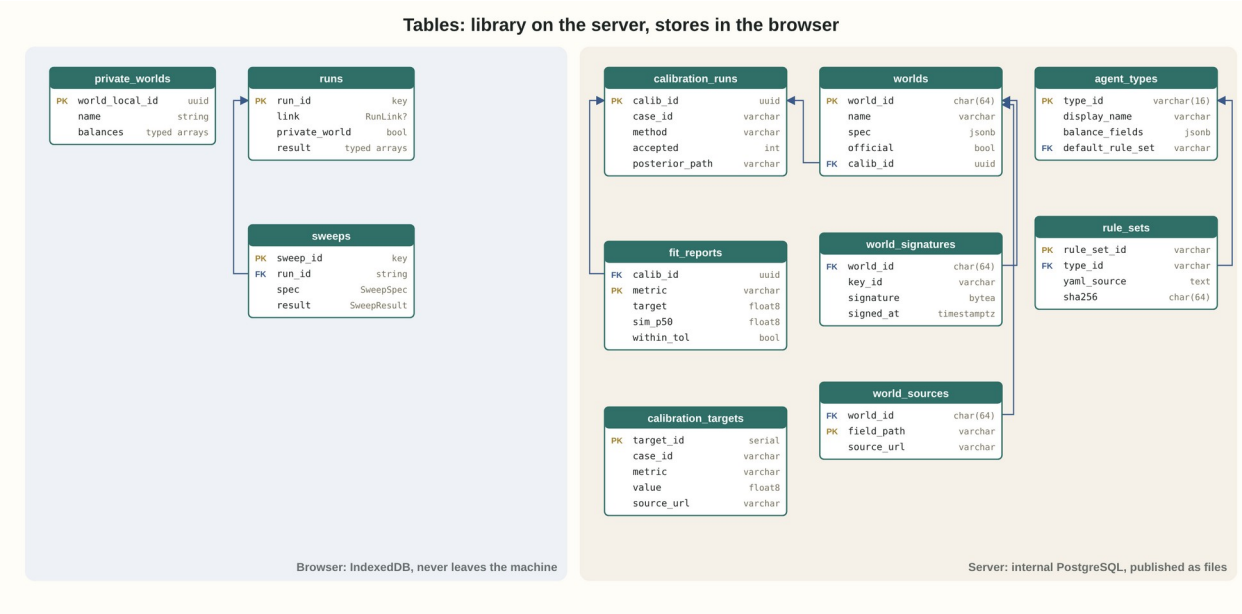


Thread	How many	What it does	How it synchronises
Main thread	One	Interface, loading, drawing, IndexedDB	Never blocks: everything is async
WorkerPool	Part of the main thread	Distributes segments, collects results, resends a failed segment	A Promise per segment
Worker 1..N	navigator.hardwareConcurrency minus 1	Pyodide, NumPy, the engine	Messages only
Dask scheduler + workers	In the calibration cluster	ABC-SMC rounds	Dask futures

Three rules

- **No shared memory, so no locks:** no SharedArrayBuffer and no Atomics. Each thread holds its own data, and ownership moves with the message.
- **The seed comes from the segment index:** seed = hash(base_seed, batch_idx). So the sweep gives the same result even if Worker 3 finishes before Worker 1, and even if a Worker crashed and the segment was rerun elsewhere.
- **One core stays free:** the pool leaves a core for the main thread, so the interface does not freeze during a sweep.

Table design



Two separate places, with no connection between them. On the server, an internal PostgreSQL from which signed static files are exported. In the browser, IndexedDB that never leaves the machine.

On the server (internal, published as files)

Table	What is stored	Note
agent_types	The five agent types and the balance-sheet fields of each	LDI, bank, PCF, insurer, MMF
rule_sets	The YAML file of each rule set, with a hash	A new version is a new row, not an update
worlds	A world definition, official or not, and which calibration fed it	world_id is the hash of the content
world_signatures	Signature, key identifier, time	A world without a signature is never marked official
world_sources	A source for every field in the world	Every number in an official world has a link
calibration_targets	Target data from the historical case (for example the gilt yield move in Britain, September 2022)	With a source
calibration_runs	A calibration run: method, number accepted, posterior path	
fit_reports	For each metric: target, simulated median, within tolerance or not	Anything out of tolerance is published in red

In the browser (IndexedDB)

Store	What is stored	Note
runs	Definition, link, result as typed arrays	link empty for a private world
sweeps	Sweep definition and result	Belongs to a run
private_worlds	Balance sheets the user uploaded	purgePrivate() deletes everything

Store	What is stored	Note
		in one click

Implementation order by versions

The same rule as Case 01: first the core, and only then what wraps it. The versions match the sprints in the architecture.

Version	What is built	Exit test
V0.1	CascadeEngine, Balances, AssetState, two agent types, conservation checks, in plain Python	Reproduces the POC numbers: 0.6 becomes 1.125
V0.2	RuleCompiler with YAML, the three impact models, all five agent types	The same rule set in YAML and in code gives an identical result
V0.3	Pyodide inside a Worker, WorldLoader, a basic AppShell	The same seed in the browser and plain Python: identical to the bit
V0.4	WorkerPool, SweepRunner, SensitivityAnalyzer, tornado	512 worlds on 8 cores in under 10 seconds; a Worker killed midway does not change the result
V0.5	ContagionNet, DebtRank, the four interventions	Intervention off: identical to V0.4. Central-bank purchases: the cascade stops
V1.0	Signed library, calibration, LinkCodec, LocalStore	A link opened on another machine reproduces the same run; a world whose signature was altered is marked unofficial

What this document closes, and what it does not

After the design document, the engine is defined down to the level of the class and the message, and every promise from the spec (reproducibility, a range, privacy of a private world) is translated into a concrete structure.

And what is not closed, here too, is the same thing the architecture's closure check found: the worlds table has a spec field, and inside it the market's average leverage. In an official world it comes from calibration on a historical case. For today's market it has no source. Design cannot solve this, because it is not a design problem.